

You own identity. Every other group's work is meaningless if the server cannot say who is asking.

YOU OWN `backend/core/views.py (auth)` `backend/core/serializers.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `POST /api/auth/login/` — authenticate with `django.contrib.auth`, never a plaintext comparison. Return access + refresh + the user object exactly as the contract shows.
- ☐ `POST /api/auth/refresh/` — SimpleJWT gives you this almost free.
- ☐ `GET /api/auth/me/` — return `request.user`. Five lines, and the whole point: a token you cannot resolve back to a user is not authentication.
- ☐ A management command to create student logins from the roster, with roll number as username.
- ☐ Mark `login` and `refresh` as `AllowAny`. Everything else stays locked.

WAITS ON `NEURALTEAM`

WAITED ON BY `GRP8` `GRP7` `CLAUDE`

Done looks like: `curl -X POST ../login/` with a good roll number returns two tokens; a bad password returns 401; `/me/` with the access token returns that student.

You own the one request the whole system exists to serve. It is also the one everybody will try to cheat.

YOU OWN `backend/core/views.py (mark)` `backend/core/serializers.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `POST /api/attendance/mark/`, multipart. Take the student from `request.user` — **never** from a `user_id` field the client sent.
- ☐ Stamp `marked_at` on the **server**. A client-supplied timestamp lets anyone backfill last Tuesday.
- ☐ Reject: already marked today (409), Sunday, a `Holiday` row, outside class hours.
- ☐ Add the `UniqueConstraint` on (student, session) and wrap the write in `transaction.atomic()`. A check-then-write is a race, not a constraint.
- ☐ Call `presence.check_campus_ip()` and `presence.verify_session_code()` as gates; call `faceid.verify()` for a score. Store lat/lng but **never gate on them**.

WAITS ON `NEURALTEAM` `GRP5` `BHUVANESHWARI` `BALAJI`

WAITED ON BY `DRAGONS`

Done looks like: A valid mark returns 201. The same student twice in one day returns 409. No token returns 401. Spoofed coordinates still return 201 — and you can explain why.

You own everything the app reads back. Three endpoints, and the Django admin nobody else will build.

YOU OWN `backend/core/views.py` (reads) `backend/core/admin.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `GET /api/attendance/today/` — has this student marked, and the record if so.
- ☐ `GET /api/attendance/my-records/?year=&month=` — the month's records plus a summary with the percentage. Exclude Sundays and holidays from the denominator; a rate computed over calendar days is wrong and the reference app gets this wrong twice.
- ☐ `GET /api/admin/overview/?date=` — staff only, with a per-student status list. Return 403 for a student, not an empty list.
- ☐ Register **every** model in `admin.py`. The reference registers one of ten and you cannot inspect anything.
- ☐ A `seed_demo` management command with a **fixed random seed**, so every team's numbers match the projector.

WAITS ON **NEURALTEAM** **GRP5**

WAITED ON BY **GRP14** **ASTERIX**

Done looks like: `seed_demo` then `/api/admin/overview/?date=...` returns counts that match what the seed script printed.

You unblock the entire class. Nothing else runs until your first commit lands.

YOU OWN `backend/core/urls.py` `backend/config/settings.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Replace the stub `backend/core/urls.py` with routes for all eight endpoints, each pointing at a view the other BE groups will write. Import them lazily or stub the views with `pass` so the file imports cleanly before their work lands.
- ☐ `pip install djangorestframework-simplejwt django-cors-headers`.
- ☐ Add `REST_FRAMEWORK` with `DEFAULT_AUTHENTICATION_CLASSES = JWTAuthentication` and `DEFAULT_PERMISSION_CLASSES = IsAuthenticated`. Default to locked, open individual views deliberately.
- ☐ Add `SIMPLE_JWT` (access 30 min, refresh 1 day) and `CORS_ALLOWED_ORIGINS` for `http://localhost:5173`.
- ☐ `python manage.py makemigrations core && migrate` — the first migration for the nine existing models.

WAITS ON **nothing — start now**

WAITED ON BY **GRP5** **GRP7** **CLAUDE**

Done looks like: `manage.py migrate` creates the `core_*` tables and `runserver` starts with all eight routes registered, even if seven return 501.

You own the screen this entire course is about. Thirty seconds, one thumb, standing up.

YOU OWN

`frontend/src/app/components/employee/MarkAttendance.tsx`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Port camera + geolocation from `pmutwd_attendance/fe/.../MarkAttendance.tsx:224–332` and **nothing else** from that 1,285-line file.
- ☐ Keep the two patterns it earned the hard way: attach the stream in a `useEffect` (the `<video>` has not mounted when the stream arrives), and retry geolocation with `enableHighAccuracy:false` on failure.
- ☐ Keep both capture artefacts: a `dataURL` for the preview, a `File` for the upload.
- ☐ An input for the six-character projector code.
- ☐ Handle every failure the contract lists: 403 with a readable reason, 409 already marked, and no-camera-permission.

WAITS ON

GRP8

GRP7

MINIONS

AGENTS

MINIBYTES

WAITED ON BY

nobody is waiting on you — you set your own pace

Done looks like: On a real phone over HTTPS: camera opens, photo captures, coordinates appear, submit returns 201. Second submit the same day shows a human-readable 409.

You own what staff see. One question: who is here today?

YOU OWN

`frontend/src/app/components/admin/AdminDashboard.tsx`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Date picker, defaulting to today, calling `/api/admin/overview/?date=`.
- ☐ A recharts donut — present / absent / not marked — with live counts beside it.
- ☐ A searchable student list with filter tabs, each tab showing its own count.
- ☐ Staff only. A student who reaches this URL gets sent away, not shown an empty page.
- ☐ Build against the contract with mock data first. CLAUDE's endpoint will arrive later than you want it to.

WAITS ON

GRP8

CLAUDE

GRP4

WAITED ON BY

nobody is waiting on you — you set your own pace

Done looks like: Pick a seeded date and the donut, the counts and the list all agree with what `seed_demo` printed.

You own the plumbing every other frontend group builds on. Get it right and they never think about auth again.

YOU OWN `frontend/src/app/services/api.ts` `frontend/src/app/context/AuthContext.tsx`
`frontend/src/app/components/ProtectedRoute.tsx`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Finish `services/api.ts` : typed functions for all eight endpoints, matching the contract page exactly.
- ☐ The **401** → **refresh** → **retry** interceptor. Without it everyone is silently logged out after 30 minutes and it will look like a backend bug.
- ☐ Make `AuthContext` real. Keep the access token in memory, not `localStorage` — and be able to say why in your pull request.
- ☐ Fix `ProtectedRoute` so it stops bouncing everything to `/login` once a user exists.
- ☐ **Do not wait for the backend.** Build against the contract with a mock, and swap the base URL when GRP5 lands.

WAITS ON **GRP5**

WAITED ON BY **DRAGONS** **ASTERIX** **GRP14**

Done looks like: Log in on a real backend, land on `/mark`, refresh the page and stay logged in. Let the access token expire and watch the next call succeed anyway.

The first screen every student sees, and the one they will actually keep opening.

YOU OWN `frontend/src/app/components/Login.tsx` `frontend/src/app/components/MyRecords.tsx`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `Login` : roll number and password, two controlled inputs. Show the server's error on a 401 — “Something went wrong” helps nobody.
- ☐ Send students to `/mark` and staff to `/admin` after login.
- ☐ `MyRecords` : month picker calling `/api/attendance/my-records/`.
- ☐ Calendar view and list view with a toggle, using GRP4's `MonthCalendar` and `AttendanceRow`.
- ☐ Show the running percentage prominently. It is the number students actually care about.

WAITS ON **GRP8** **GRP5** **CLAUDE** **GRP4**

WAITED ON BY nobody is waiting on you — you set your own pace

Done looks like: Log in as a seeded student and see a month of real attendance with a percentage that matches the API's summary.

The three components with real logic in them. The calendar is the hardest thing in the frontend.

YOU OWN `frontend/src/app/components/ui/MonthCalendar.tsx` `.../AttendanceRow.tsx` `.../StatCard.tsx`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `MonthCalendar` — seven-column grid, correct leading blanks for the first weekday, one cell per day coloured by status, plus a legend.
- ☐ Colour priority when a day is several things at once: present > leave > holiday/Sunday > past-unmarked > future. Today gets a ring.
- ☐ `AttendanceRow` — one record in a list: date, time, status, and buttons for photo and map.
- ☐ `StatCard` — a count with a label, used four times on the dashboard.
- ☐ Take the five status names from NEXAAI's contract. Do not invent your own.

WAITS ON `NEXAAI` `MINIONS`

WAITED ON BY `GRP14` `ASTERIX`

Done looks like: Render a month with a mixture of present, absent, holiday and future days, and every cell is the colour the contract says it should be.

You own the vocabulary. Get the tokens right and nobody ever hardcodes a colour.

YOU OWN `frontend/src/styles/theme.css`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Replace the placeholder palette with a real Basar one. Do not copy the reference app's.
- ☐ Typography scale, spacing scale, radii — using the Tailwind v4 `@theme inline` pattern already in the file.
- ☐ Dark mode via the `.dark` block. It is nearly free if the tokens are right, and it is the test of whether they are.
- ☐ Check contrast. This gets used on a phone, outdoors, in daylight.

WAITS ON `NEXAAI`

WAITED ON BY `GRP4` `AGENTS` `DRAGONS` `GRP14` `ASTERIX`

Done looks like: Change one value in `theme.css` and watch the colour move on all four screens. If you have to edit a component to change a colour, you are not done.

Three shared components. Every one of them is copy-pasted two or three times in the reference app.

YOU OWN

frontend/src/app/components/ui/AppHeader.tsx

.../PhotoModal.tsx

.../EmptyState.tsx

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ AppHeader — title, the logged-in student, a logout button. The reference re-implements this inline on every screen.
- ☐ PhotoModal — full-screen photo view with a close control. Must work with a thumb.
- ☐ EmptyState — nothing to show yet. Nobody ever designs this one, and every screen needs it.
- ☐ Read from tokens, never hardcode a colour. Props typed, no any .

WAITS ON

NEXAAI

MINIONS

WAITED ON BY

DRAGONS

ASTERIX

GRP14

Done looks like: All three imported and rendering in at least two different screens, with zero hardcoded hex values.

Two people, one decision that everybody else obeys. Small scope, high leverage.

YOU OWN

frontend/src/styles/theme.css (status block)

docs/ui-contract.md

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Decide the five status colours once: present, absent, leave, holiday, today. Write them into theme.css as tokens.
- ☐ Write docs/ui-contract.md : the token name for each state, when each is used, and what the legend text says.
- ☐ Hand it to the three FE groups and the other three UI/UX groups. This is the deliverable — the handoff, not the code.
- ☐ Do this **first** and fast. GRP4 and AGENTS cannot finish their components until the names exist.

WAITS ON

nothing — start now

WAITED ON BY

GRP4

AGENTS

MINIONS

GRP14

ASTERIX

Done looks like: Every other group can name the token for “present” without asking you.

You put the LLM in it, then prove it works. The last three days of the course are yours.

YOU OWN `backend/llm/client.py` `backend/agent/loop.py` `backend/evals/run.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `llm/client.py` — call the class proxy with `claude-haiku-4-5-20251001`. Report cost per call in rupees.
- ☐ `POST /api/ask/` — plain-English question in, **schema-valid JSON** out, dispatched to one of FEMMEFORCE's query functions.
- ☐ `agent/loop.py` — the tool loop, written by hand, in an empty file. No framework.
- ☐ Tool schemas that wrap the query functions and nothing else. That boundary *is* the security model.
- ☐ `evals/run.py` — at least 20 real questions, a before-score, an after-score, and a paragraph on what changed.

WAITS ON **FEMMEFORCE**

WAITED ON BY nobody is waiting on you — you set your own pace

Done looks like: The agent answers a question needing two tool calls, and `python -m evals.run` prints a score that moved.

You turn a face into 512 numbers. Everything else on this track is downstream of you.

YOU OWN `backend/faceid/embed.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Load the ONNX detector and MobileFaceNet/ArcFace models from `backend/models/`. Three pure wheels, no compiler.
- ☐ `embed(image_bytes) -> np.ndarray` of shape (512,) — detect, crop, align, normalise, run.
- ☐ `cosine(a, b) -> float`. Ten lines, and every threshold decision rests on it.
- ☐ Handle no-face-found and more-than-one-face explicitly. Do not return a vector for an empty frame.
- ☐ Time it. Report milliseconds per image to the class — it decides whether this can run inline or must go to a queue.

WAITS ON **nothing — start now**

WAITED ON BY **BALAJI**

Done looks like: Two photos of the same person score above 0.6; two different people score below. You can state your numbers.

You pick a number that decides whether a student sits an exam. That is a product decision, not a technical one.

YOU OWN `backend/faceid/verify.py` `backend/faceid/sweep.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Enrolment: store ten reference photos per student as `EnrolmentPhoto` rows. `set_vector()` already exists on the model.
- ☐ `verify(roll_no, image_bytes) -> float` — best cosine against that student's enrolled vectors.
- ☐ `sweep.py` — sweep the threshold across all enrolled photos and plot false-accept against false-reject.
- ☐ **Choose your threshold from that curve and be ready to defend it out loud.**
- ☐ Liveness: two frames 800 ms apart, reject if byte-identical or cosine ≈ 1.000 . Then hold a photo of a face to the camera and watch it pass anyway.

WAITS ON **ANUSHA**

WAITED ON BY **GRP7**

Done looks like: Your own face passes, your partner's fails, and you can say: "I chose 0.62 because a false reject costs a student their exam eligibility and a false accept costs the institution very little."

Four functions. They are the only Python the LLM will ever be allowed to run.

YOU OWN `backend/core/queries.py` `backend/tests/test_queries.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `attendance_percentage(roll_no)` — worked example, given to you.
- ☐ `students_below_threshold(threshold) -> [{roll_no, name, percentage}]`.
- ☐ `attendance_by_subject(roll_no)` and one more of your choosing.
- ☐ Exclude Sundays and holidays from the denominator. A percentage over calendar days is wrong, and the reference app gets it wrong in two places.
- ☐ Write the tests. `pytest backend/tests/test_queries.py` → 4 passed.

WAITS ON **CLAUDE**

WAITED ON BY **GRP11**

Done looks like: Four tests pass and the at-risk count equals the number on the projector.

You decide what counts as proof someone was in the room. Two gates and one deliberate non-gate.

YOU OWN `backend/presence/signals.py`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ `check_campus_ip(request)` against `CAMPUS_CIDR`. Read `X-Forwarded-For`, **not** `REMOTE_ADDR` — behind Caddy that is the proxy's address, and every student would look identical.
- ☐ `verify_session_code(code, student)` — 60-second TTL, single use per student via `CodeRedemption`.
- ☐ A view that shows a rotating code on the projector.
- ☐ `haversine()` — write it, test it, log the distance, and **never wire it into the accept path**. Be ready to explain to someone who thinks that is a bug.

WAITS ON **nothing — start now**

WAITED ON BY `GRP7`

Done looks like: The projector code returns 200. The same code 90 seconds later returns 403. Spoofed coordinates still return 200 — and you can explain that to the room.

You protect twenty groups from each other. Nobody notices CI until it saves them.

YOU OWN `.github/workflows/ci.yml` `.github/workflows/deploy.yml`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ CI on every pull request into `dev`: `pytest`, `npm run typecheck`, `npm run build`.
- ☐ A secret-scanning step. `git log -p --all | grep -c 'sk-ant'` must return 0, and the reference repo committed live AWS keys.
- ☐ Fail the build on a type error. `tsc --noEmit` alone silently checks nothing with this config — it must be `tsc -b --noEmit`.
- ☐ Deploy on merge to `main`. The reference pipeline is `git pull && migrate && restart` with no tests and no rollback. Improve on it and say why in the PR.

WAITS ON **nothing — start now**

WAITED ON BY nobody is waiting on you — you set your own pace

Done looks like: Open a pull request with a deliberate type error and watch CI reject it before a human looks.

Without you there is no camera and no GPS. You are on the critical path and most people will not realise it.

YOU OWN `backend/deploy/Caddyfile` `backend/deploy/*.service` `verify.sh`

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Caddy with automatic HTTPS, one subdomain per group. **Do this early.**
- ☐ `getUserMedia` and the Geolocation API only work in a secure context. Over plain HTTP `navigator.mediaDevices` is `undefined` — so DRAGONS gets a `TypeError` that reads exactly like their own bug.
- ☐ A systemd unit that survives `sudo reboot` without anyone touching it.
- ☐ `.env` handling, and a `verify.sh` that checks the service is up and the cert is valid.
- ☐ Serve the built frontend from Caddy so there is one origin in production.

WAITS ON **nothing — start now**

WAITED ON BY **DRAGONS**

Done looks like: `curl -I https://<team>.<domain>/` returns 200 with a valid cert, a classmate opens it on their phone, and it survives a reboot.

You make the system accountable. The rejected attempts are the interesting half.

YOU OWN `backend/presence/audit.py` `backend/core/admin.py` (audit)

SPRINT 1 — DO THESE, IN THIS ORDER

- ☐ Write an `AuditLog` row on **every** attempt — accepted or rejected — with the reason, IP, and coordinates.
- ☐ Never let an audit failure block a legitimate mark. Log and continue.
- ☐ A staff-only view listing today's rejections with reasons.
- ☐ Redact nothing that matters and store nothing that does not. Coordinates are recorded for audit and never used to decide.

WAITS ON **GRP7**

WAITED ON BY nobody is waiting on you — you set your own pace

Done looks like: Attempt a mark from off-campus, get rejected, and find the row in the audit log with the reason.